# Sheth L.U.J. And Sir M.V. College

Aim:Implementation of Events, Commands, and Tasks in TinyOS
Develop a TinyOS program that toggles LEDs using events and tasks to demonstrate split-phase execution.

**Source Code**

**1. Module Implementation (`BlinkC.nc`)**

Implements the core application logic, timer events, commands, and deferred task execution.
Code:

```
#include "Timer.h"
module BlinkC @safe() {
  uses interface Timer<TMilli> as Timer0;
  uses interface Leds;
  uses interface Boot;
}
implementation {
  uint16_t counter = 0;

  // TASK: Deferred background execution (Split-Phase Phase 2)
  task void toggleLedTask() {
    counter++;
    dbg("BlinkC", "TASK RUNNING: Processing split-phase task, counter =
%d\n", counter);

    // COMMAND: Call LED driver to toggle LED0
    call Leds.led0Toggle();

    // Toggle LED1 on every alternate execution
    if (counter % 2 == 0) {
      call Leds.led1Toggle();
      dbg("BlinkC", "TASK: LED1 toggled on even count.\n");
    }
  }
  // EVENT: System booted
  event void Boot.booted() {
    dbg("BlinkC", "BOOT EVENT: System booted, starting periodic timer.\n");
    // COMMAND: Start 250ms periodic timer
```

MITHIL KADAM T083

```
    call Timer0.startPeriodic(250);
  }
  // EVENT: Hardware timer tick (Split-Phase Phase 1)
  event void Timer0.fired() {
    dbg("BlinkC", "EVENT: Timer0 fired! Posting task to TinyOS scheduler...\n");
    // Post task to scheduler queue to avoid blocking interrupt context
    post toggleLedTask();
  }
}
```

### 2. Configuration & Wiring (`BlinkAppC.nc`)

Wires the application module interfaces to system components (`MainC`, `LedsC`, and `TimerMilliC`).

Code:

```
configuration BlinkAppC {
}
implementation {
  components MainC;
  components LedsC;
  components BlinkC as App;
  components new TimerMilliC() as Timer0;

  // Wiring interfaces
  App.Boot -> MainC.Boot;
  App.Leds -> LedsC;
  App.Timer0 -> Timer0;
}
```

MITHIL KADAM T083

### 3. Simulation Script (`sim.py`)

Configures the TOSSIM simulation engine, initializes node 1, routes debug channels to standard output, and executes simulation events.

```
import sys
from TOSSIM import *

t = Tossim([])
r = t.mac()

# Route debug channels
t.addChannel("BlinkC", sys.stdout)
t.addChannel("LedsC", sys.stdout)

# Boot mote 1 at time 0
m = t.getNode(1)
m.bootAtTime(0)

print("=== Starting Split-Phase Execution Simulation ===")

# Run 100 simulation events
for i in range(100):
    t.runNextEvent()

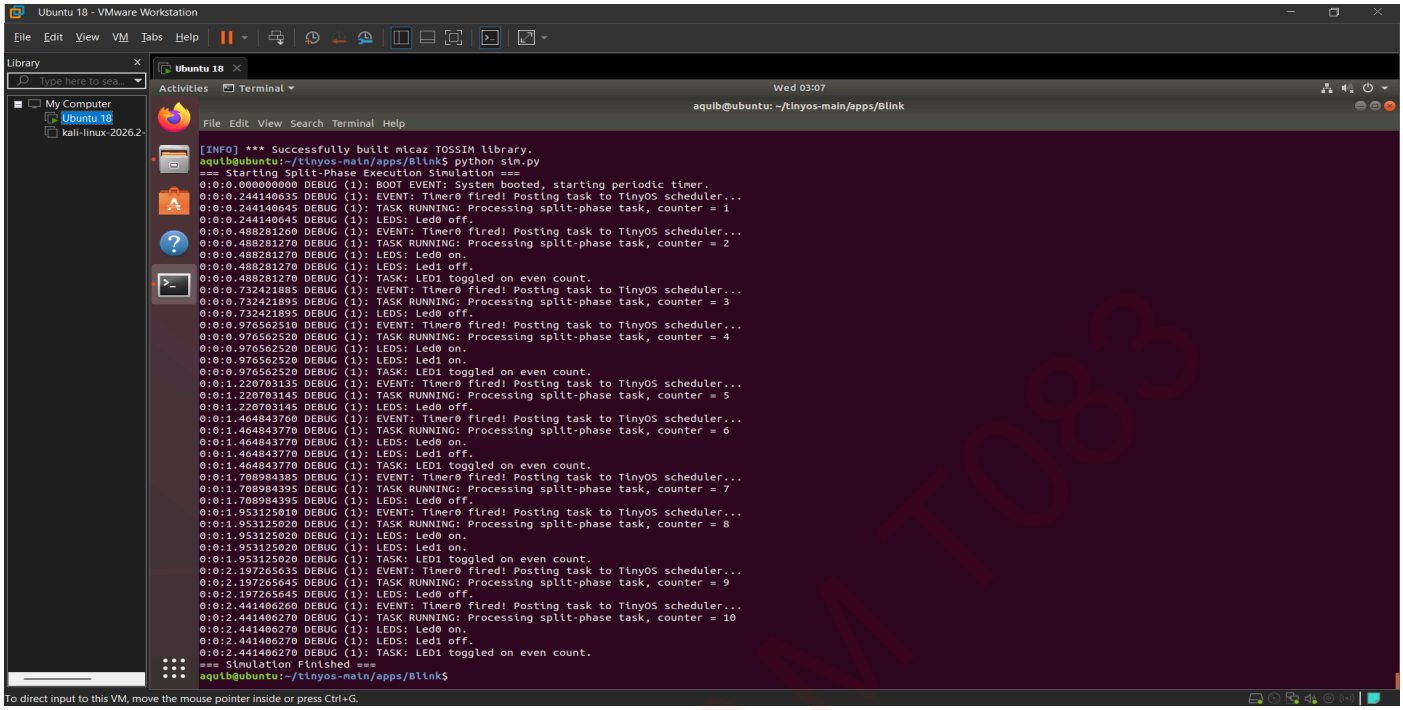
print("=== Simulation Finished ===")
```

MITHIL KADAM T083

**OUTPUT:**

Name :- Mithil Kadam
Roll No. T083
Subject:- Wireless And Sensor Networks

# Sheth L.U.J. And Sir M.V. College




MITHIL KADAM T083

Name :- Mithil Kadam
Roll No. T083
Subject:- Wireless And Sensor Networks